

Lab 4: Fourier Transforms and Hybrid Images

Due Wednesday, February 25, 11:59pm

Images can be thought of in two equivalent ways: as a spatial arrangement of pixel intensities, or as a combination of sinusoidal patterns at different frequencies. Low frequencies correspond to smooth, gradual changes in an image, while high frequencies correspond to sharp edges and fine details. The Fourier transform converts between these two representations. Understanding the frequency content of images is fundamental to computational imaging. This principle underlies image filtering, compression, and reconstruction algorithms used throughout this course.

In this lab, you will compute and interpret Fourier transforms of images, implement frequency-domain filtering to remove or enhance specific features, explore aliasing and why anti-aliasing filters are necessary, and create hybrid images that appear differently when viewed from different distances.

Allowed libraries. You may use numpy, scipy, matplotlib, and imageio.

1 Fourier Transform and Filtering

The 2D Discrete Fourier Transform (DFT) decomposes an image into sinusoidal components at different frequencies and orientations. For an $M \times N$ image, the DFT produces an $M \times N$ complex-valued array where each entry represents the amplitude and phase of a particular frequency component. After applying `fftshift`, the center of the spectrum represents low frequencies, and the edges represent high frequencies.

Problem 1.1: Which has on average higher frequency content: an image I or $\text{blur}(I)$?

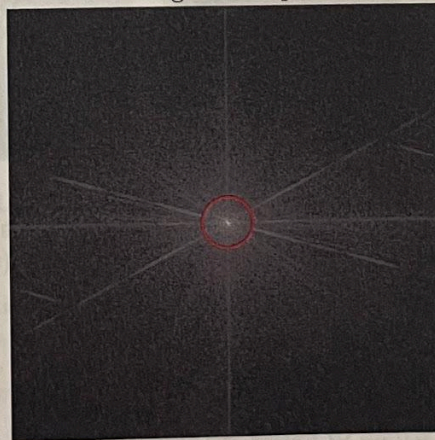
The image I has an average higher frequency content, since it would have sharper edges, and thus less high frequency components

Problem 1.2: Below is an image and its Fourier transform. Find the corresponding images if we reconstructed it using inverse FFT after (a) masking the inside of the red circle and (b) masking the outside of the red circle.

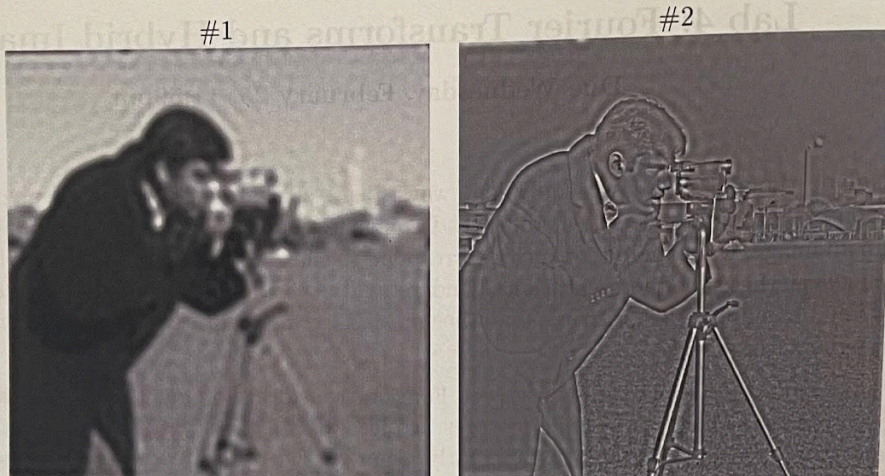
Original Image



FFT Magnitude Spectrum

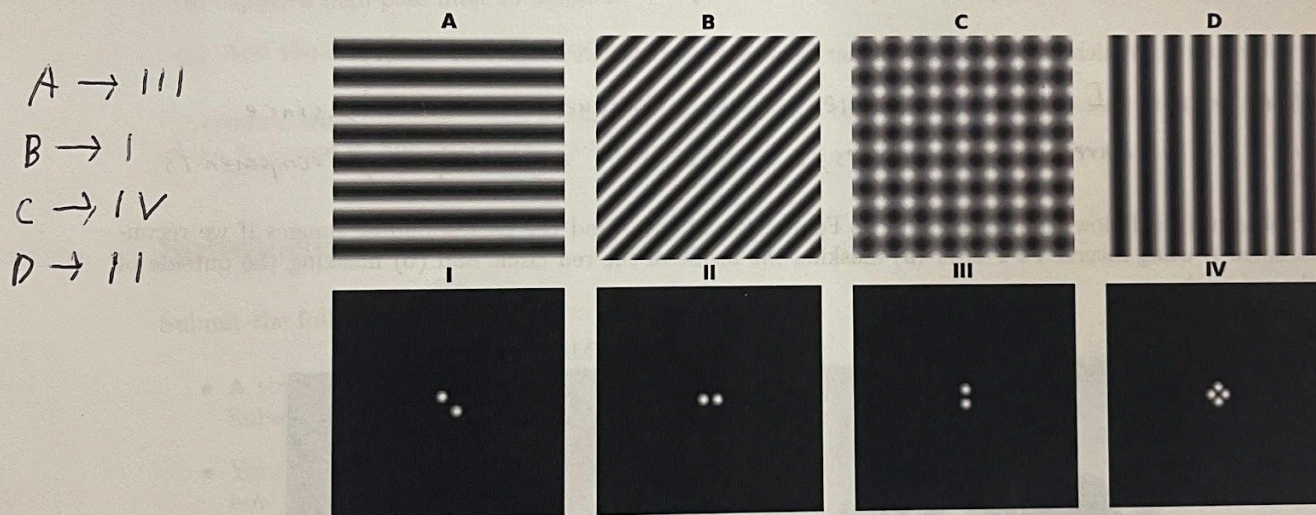


- a) Masking the inside results in image 1 (only keep low frequency)
- b) Masking the outside results in image 2 (only keep high frequency)



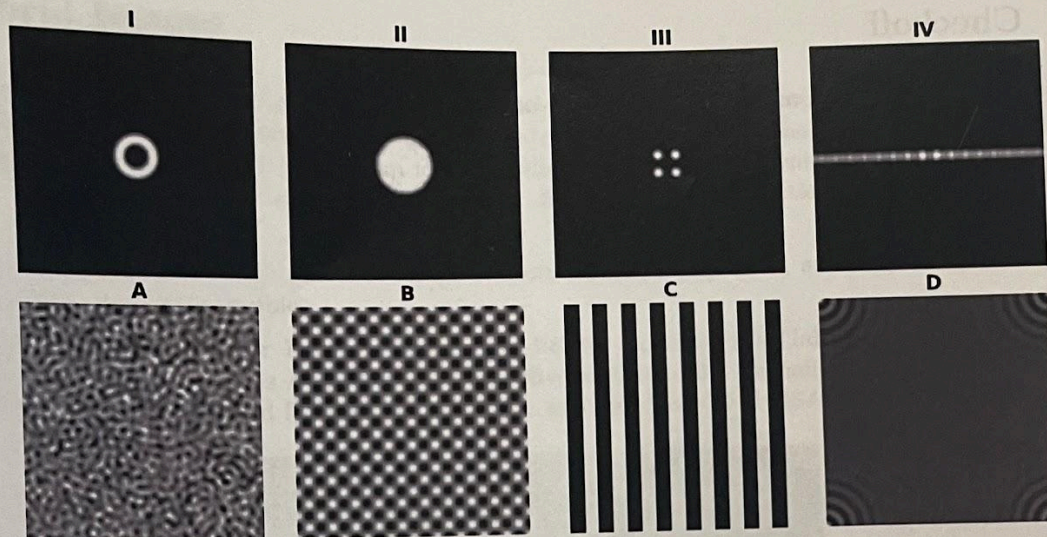
Problem 1.3: A pure 2D sinusoid at a single frequency and orientation produces exactly one pair of symmetric dots in the Fourier magnitude spectrum. The dots' position encodes both the frequency (distance from center) and the orientation (angle) of the sinusoid.

- (a) The top row below shows four spatial-domain patterns (A–D) and the bottom row shows their log-magnitude spectra in shuffled order (I–IV). Match each pattern to its spectrum.



- (b) The top row shows Fourier magnitude spectra of four images (I–IV) and the bottom row shows the images in shuffled order (A–D). Unlike (a), these are not pure sinusoids. Match each spectrum to its image.

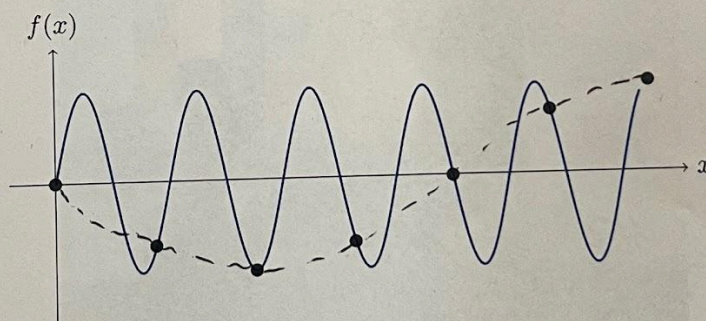
$A \rightarrow I$
 $B \rightarrow III$
 $C \rightarrow IV$
 $D \rightarrow II$



Problem 1.4: Suppose you downsample an image by a factor of 2. What is the highest frequency that this new image can represent in terms of the previous highest frequency?

The new image has at most half the previous highest frequency

Problem 1.5: Consider this 1D signal sampled at regular intervals:



What lower-frequency signal would you reconstruct if you connected these sample points? Sketch it below. Why might this be a problem when downsampling an image?

When you downsample too much, you lose information, and the remaining sample could lead you to reconstructing the wrong signal

Problem 1.6: In Lab 3, you implemented 2D convolution from scratch using nested loops. The convolution theorem states that convolution in the spatial domain is equivalent to element-wise multiplication in the frequency domain:

$$I * K = \mathcal{F}^{-1}(\mathcal{F}(I) \cdot \mathcal{F}(K))$$

Implement a function that performs convolution using FFTs. Zero-pad the kernel to match the image size before transforming. Test your implementation by convolving an image with a 9×9 box blur kernel and comparing the result to `scipy.ndimage.convolve`. Using your loop-based `convolve2d` implementation from Lab 3, compare runtime for kernel sizes 3, 11, and 21 on the full-size image. At what kernel size does the FFT approach become faster?

The FFT method was faster for all 3 kernel tests.

Checkoff

When you're ready, ask a TA to check you off.

1 ← A
111 ← B
V1 ← C
11 ← D

2 Hybrid Images

A hybrid image is a single image that changes interpretation with viewing distance: up close, high-frequency details dominate, and from far away the low-frequency structure dominates. The standard recipe is to align two images, low-pass filter one (e.g., with an ideal circular cutoff in the frequency domain), form the high-pass of the other, and add them together. The cutoff is chosen by experimentation so the two bands do not compete.

This section is intentionally visual: include image outputs wherever requested, and keep images reasonably sized so the PDF stays manageable.

This idea shows up in Salvador Dalí's 1976 painting below—up close it's Gala looking at the sea, but from across the room it becomes Abraham Lincoln. Oliva and Torralba formalized this technique in their SIGGRAPH 2006 paper "Hybrid Images" (<https://dl.acm.org/doi/10.1145/1179352.1141919>).

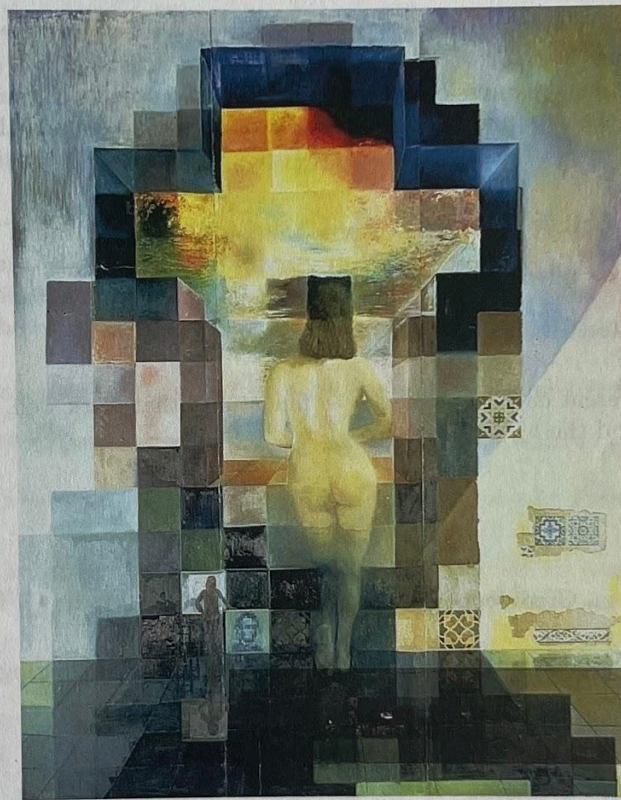


Figure 1: Salvador Dalí, *Gala Contemplating the Mediterranean Sea Which at Twenty Meters Becomes the Portrait of Abraham Lincoln* (1976).

Problem 2.1: Explain why hybrid images work.

Hybrid Images work because it separates the high and low frequency parts of the image generally into two, where when you are close, you can see the higher frequency parts of the image (the woman), and when you are farther away, you can only see the lower frequency parts. This is because your eye, as distance increases, will act more and more like a low pass filter, since it can't make out the high frequency parts.

Problem 2.2: Write a function to compute and visualize the 2D FFT of an image. Your function should:

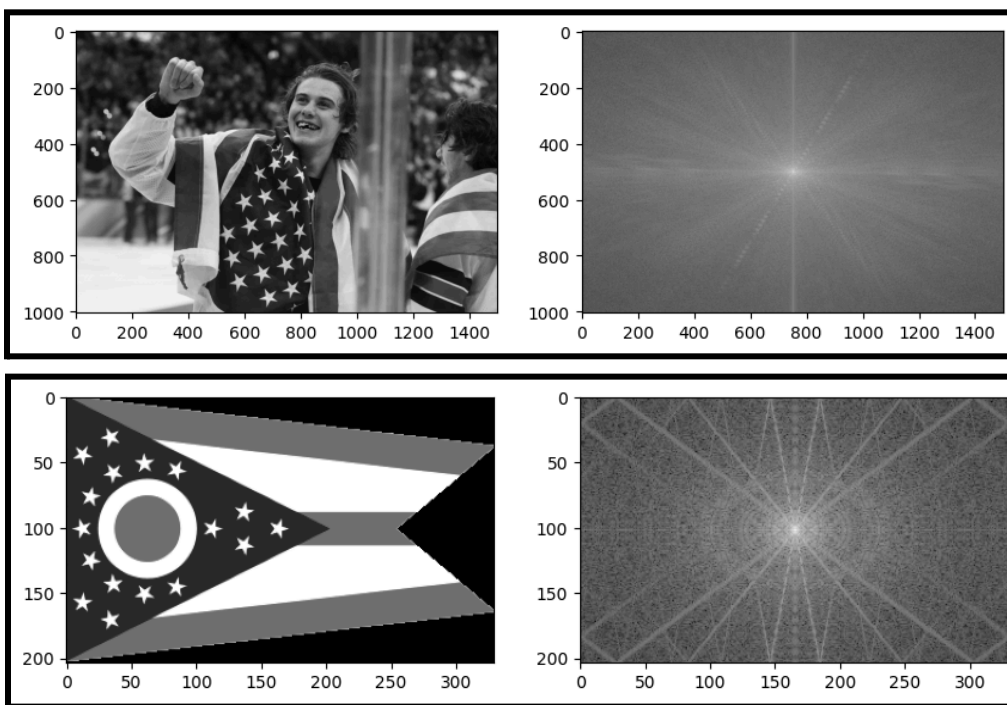
- (a) Compute the FFT using `numpy.fft.fft2`.
- (b) Shift the zero-frequency component to the center using `fftshift`.

CS 4660: Foundations of Computational Imaging

Spring 2026

- (c) Display the log-magnitude spectrum.

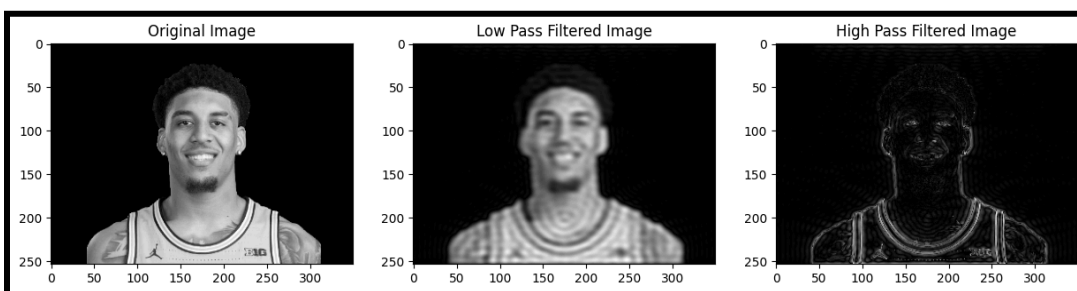
Apply your function to at least two different images and show the results (images plus spectra).



Problem 2.3: Implement low-pass and high-pass filters in the frequency domain. Create functions that:

- (a) Take an image and a cutoff frequency as input.
- (b) Apply an ideal circular mask in the frequency domain (hard cutoff).
- (c) Return the filtered image.

Include an image showing the original, low-pass, and high-pass results.



Problem 2.4: Create a hybrid image by combining the low-frequency content of one image with the high-frequency content of another. Use two images of your choice that align well (e.g., two faces that have different details like glasses or facial expressions, two animals, or two objects of similar shape). The most important part is to make sure the eyes match. Include the two source images and the final hybrid image. Your code should:

- (a) Apply a low-pass filter to image A.
- (b) Apply a high-pass filter to image B.
- (c) Add the two filtered images together.

Experiment with different cutoff frequencies to find values that work well, and include a figure that shows image A, image B, and the hybrid result.

